

1. 为什么 HDFS 适合大文件而不适合小文件？

HDFS 面向大文件顺序读写和高吞吐访问。它把文件切成大 Block，分布存储在多个 DataNode 上。NameNode 需要维护文件、Block、DataNode 之间的元数据。如果小文件很多，每个小文件都会占用 NameNode 元数据空间。同时小文件会带来大量任务调度和寻址开销，无法发挥顺序读写优势。所以 HDFS 适合少量大文件，不适合海量小文件。

2. 为什么 HDFS 的 Block 比单机文件系统的 Block 大？

单机文件系统的 Block 通常较小，是为了兼顾小文件和随机读写。HDFS 的 Block 较大，是因为它主要处理大文件和顺序读写。大 Block 可以减少寻址次数，降低磁盘寻道开销。同时也能减少 Block 数量，从而减少 NameNode 维护的元数据。这样更适合大规模批处理任务，提高整体吞吐量。所以 HDFS 用大 Block 是为了高吞吐，而不是低延迟。

3. HDFS 写文件流程是什么？

Client 先向 NameNode 请求创建文件。NameNode 检查权限和路径，并返回可写入的 DataNode 列表。Client 把文件切成 Block，并按副本策略建立写入 pipeline。数据先写入第一个 DataNode，再依次转发给后续副本节点。所有 DataNode 写入成功后，会反向返回 ACK 确认。最后 Client 通知 NameNode 关闭文件，写入完成。

4. 为什么 NameNode 的 BlockMap 不持久化？

NameNode 持久化的是 FsImage 和 EditLog，用来保存文件系统目录树和操作日志。BlockMap 表示 Block 到 DataNode 的映射关系。这个映射关系和 DataNode 当前状态有关，会动态变化。DataNode 启动或周期性汇报时，会发送 Block Report。NameNode 可以根据这些汇报重新构建 BlockMap。因此 BlockMap 不需要持久化，避免保存过时信息。

5. MapReduce 的基本执行流程是什么？

MapReduce 把计算分成 Map 阶段和 Reduce 阶段。输入文件先被 InputFormat 切成 Split，每个 Split 通常对应一个 Map Task。Map 读取记录并输出中间键值对 `<k2,v2>`。Shuffle 阶段按 key 分区、排序、分组，把相同 key 送到同一个 Reduce。Reduce 对同一个 key 的 value 列表进行聚合处理。最后结果通过 OutputFormat 写回 HDFS。

6. Combiner 的作用是什么？为什么不能随便用？

Combiner 是 Map 端的本地聚合，相当于小型 Reduce。它可以在数据进入 Shuffle 前先做局部合并。这样可以减少网络传输量，提高 MapReduce 性能。Combiner 只适合满足结合律和交换律的操作，比如 sum、count、max、min。平均值不能直接对局部平均再平均，否则结果可能错误。求平均时应该传 `(sum, count)`，最后统一计算。

7. HBase 和 HDFS 有什么区别？

HDFS 是分布式文件系统，主要用于大文件顺序读写。HBase 是构建在 HDFS 上的分布式列族数据库。HDFS 适合离线批处理，不适合随机修改和实时查询。HBase 支持基于 RowKey 的随机读写、Scan 和海量稀疏表存储。HBase 的底层数据最终仍然存放在 HDFS 上。所以 HDFS 负责可靠存储，HBase 在其上提供数据库式访问能力。

8. HBase 为什么采用 LSM-tree 思想？

传统 B+ 树适合读，但随机写入时需要频繁修改磁盘页。HBase 面向海量写入场景，需要提高写性能。LSM-tree 思想是先把数据写入 WAL 和内存中的 MemStore。MemStore 达到一定大小后，再顺序写成磁盘上的 HFile。顺序写比随机写效率更高，因此能提升写入吞吐。后续通过 Compaction 合并多个 HFile，优化读取性能。

9. HBase 的读写流程分别是什么？

写入时，Client 先定位目标 RowKey 所在 Region。然后请求对应 RegionServer，数据先写入 WAL，再写入 MemStore。MemStore 达到阈值后会 flush 成 HFile，存到 HDFS。读取时，Client 同样先定位 RegionServer。RegionServer 会先查 MemStore 和缓存，再查 HFile。最后合并多个版本、删除标记和时间戳，返回结果。

10. Storm 的 Spout、Bolt、Grouping 分别是什么？

Spout 是 Storm 拓扑中的数据源，负责不断产生 Tuple。Bolt 是处理节点，负责过滤、解析、聚合、写库等操作。Tuple 是流中的一条数据记录，Stream 是 Tuple 的连续序列。Grouping 决定 Tuple 从上游节点如何发送到下游 Bolt。shuffleGrouping 适合无状态处理，用于负载均衡。fieldsGrouping 按字段分发，适合按 key 统计或维护状态。

11. Spark 为什么比 MapReduce 更适合迭代计算？

MapReduce 每轮计算通常都要读写 HDFS，中间结果大量落盘。迭代计算需要反复使用中间数据，MapReduce 会产生很大磁盘 I/O。Spark 使用 RDD 作为核心抽象，可以把中间结果缓存在内存中。多个 Transformation 会组成 DAG，遇到 Action 后统一优化执行。RDD 通过 lineage 记录依赖关系，丢失分区后可以重算恢复。所以 Spark 在迭代计算、交互分析和复杂 DAG 任务中效率更高。

12. Spark 中窄依赖和宽依赖有什么区别？

窄依赖指父 RDD 的每个 Partition 最多被一个子 Partition 使用。例如 map、filter、flatMap 通常是窄依赖，不需要 Shuffle。宽依赖指父 RDD 的 Partition 可能被多个子 Partition 依赖。例如 reduceByKey、groupByKey、join 通常是宽依赖，需要 Shuffle。Spark 会根据宽依赖划分 Stage，窄依赖可以放在同一个 Stage 流水线执行。所以判断 Stage 的关键就是看是否发生 Shuffle。

13. Flink 的 Watermark 是什么？解决什么问题？

Watermark 用来处理事件时间语义下的数据乱序问题。事件可能因为网络、队列等原因延迟到达。Watermark(t) 表示系统认为时间戳小于等于 t 的数据基本已经到齐。当 Watermark 超过窗口结束时间时，窗

口就可以触发计算。如果之后还有更早时间戳的数据到达，就属于迟到数据。因此 Watermark 让 Flink 能在乱序流中按事件时间正确开窗计算。

14. Flink 的 Checkpoint 和 Savepoint 有什么区别？

Checkpoint 是 Flink 自动周期性触发的一致性状态快照。它主要用于故障恢复，任务失败后可以从最近一次 Checkpoint 恢复。Savepoint 是用户手动触发的状态快照，通常用于运维操作。比如升级程序、迁移集群、修改并行度或暂停恢复任务。Checkpoint 通常由系统管理，Savepoint 通常由用户显式保存和使用。一句话：Checkpoint 管自动容错，Savepoint 管人工运维。

15. CAP 和 BASE 的核心思想是什么？

CAP 指分布式系统无法同时完全满足一致性、可用性和分区容忍性。在真实分布式系统中，网络分区通常不可避免，所以 P 往往必须考虑。因此系统通常需要在 C 和 A 之间取舍。BASE 是 NoSQL 系统常用思想，包括基本可用、软状态、最终一致性。它允许系统短时间内不完全一致，以换取高可用和高扩展性。所以很多 NoSQL 系统会牺牲强一致性，采用最终一致性。

综合

1. 为什么说 HDFS 是很多大数据系统的底层基础？它和 MapReduce、HBase 的关系是什么？

HDFS 负责解决大规模数据的可靠存储问题，它把大文件切成多个 Block，分散存储在不同 DataNode 上，并通过多副本机制容错。MapReduce 通常从 HDFS 读取输入数据，计算完成后再把结果写回 HDFS，所以 HDFS 是 MapReduce 的存储基础。HBase 也构建在 HDFS 之上，HBase 提供基于 RowKey 的随机读写能力，但其底层 HFile 最终仍存储在 HDFS 中。也就是说，HDFS 负责“存得下、存得可靠”，MapReduce/Spark 负责“算”，HBase 负责“随机读写访问”。

2. 为什么 MapReduce 适合离线批处理，但不适合低延迟实时计算？

MapReduce 的输入通常是 HDFS 上的静态大文件，计算过程包括任务启动、Map、Shuffle、Reduce 和结果写回 HDFS。每个 Job 启动和调度都有开销，中间结果也常伴随磁盘读写和网络传输。它更关注吞吐量，而不是单条数据的响应时间。实时计算要求数据一来就快速处理，延迟要低，而 MapReduce 的批处理模型需要积累一批数据后统一处理。因此 MapReduce 适合离线日志统计、批量分析，不适合实时告警、实时推荐这类低延迟场景。

3. Spark 相比 MapReduce 的核心改进是什么？这些改进解决了哪些问题？

Spark 的核心改进是 RDD、内存计算、DAG 执行和 lineage 容错。MapReduce 多轮作业之间通常要把中间结果写入 HDFS，磁盘 I/O 很重，迭代计算效率低。Spark 可以把中间数据缓存到内存中，并把多个 Transformation 组成 DAG，统一划分 Stage 和 Task 执行，减少不必要的落盘。RDD 通过 lineage 记录计算血统，某个 Partition 丢失后可以根据依赖关系重新计算。因此 Spark 更适合迭代计算、交互式分析和复杂多步骤数据处理。

4. HDFS、HBase、传统关系数据库分别适合什么场景？为什么不能互相完全替代？

HDFS 适合大文件顺序读写和离线批处理，不适合小文件、随机修改和低延迟查询。HBase 适合海量稀疏表和基于 RowKey 的随机实时读写，但不擅长复杂 SQL、Join 和多行事务。传统关系数据库适合结构化数据、复杂查询、事务和强一致场景，但在海量数据水平扩展、高并发写入方面不如 HBase/NoSQL 系统灵活。所以三者不是简单替代关系，而是面向不同访问模式：HDFS 管大文件存储，HBase 管大规模随机读写，关系数据库管强事务和复杂关系查询。

5. 为什么 HBase 适合 Feed 流、时序数据、日志索引这类场景？

这些场景通常有共同特点：数据量大、写入多、查询模式明确，并且常常按某个 key 或时间范围读取。HBase 的 RowKey 设计可以把相关数据组织在一起，例如 Feed 流用 `user_id` 作为 RowKey，用倒序时间戳作为列名或 RowKey 后缀；时序数据可以用 `sensor_id#reverse_timestamp` 支持按设备和时间范围扫描。HBase 底层采用 LSM-tree 思想，写入先进入 WAL 和 MemStore，再顺序写成 HFile，因此适合高吞吐写入。它不追求复杂关系查询，而是围绕访问模式设计表结构。

6. 如果让你设计一个“实时日志异常检测系统”，Storm、Spark、Flink 怎么选？

如果只是简单低延迟处理，比如日志解析、过滤、阈值报警，Storm 可以胜任，因为它的 Topology 常驻运行，Tuple 来一条处理一条。Spark 更适合批处理和微批流处理，如果对延迟要求不是极低，可以用 Spark Streaming 做周期性统计。Flink 更适合复杂实时流处理，尤其是需要事件时间、Watermark、窗口、状态和 exactly-once 语义的场景。例如“按事件时间统计 5 分钟窗口内异常次数，并处理乱序数据”，Flink 更合适。所以简单实时流可以用 Storm，批流统一分析可用 Spark，复杂有状态乱序流优先 Flink。

7. Flink 为什么比 Storm 更适合复杂有状态流处理？

Storm 主要提供 Spout、Bolt、Tuple 和 Grouping，用于构建低延迟流式拓扑，但早期对状态管理、窗口、事件时间和 exactly-once 支持较弱。Flink 是原生流处理引擎，内置 Event Time、Watermark、Window、Keyed State、Operator State 和 Checkpoint。它可以把状态交给系统管理，并通过一致性检查点在故障后恢复状态。对于按用户统计、去重、会话窗口、动态规则告警这类需要保存历史状态的任务，Flink 的表达能力和容错机制更完整。因此 Flink 更适合复杂有状态流处理。

8. Spark 的 Stage 是如何划分的？为什么宽依赖会切 Stage？

Spark 程序中的 Transformation 会形成 RDD 依赖图。窄依赖如 map、filter、flatMap 中，一个父 Partition 通常只被一个子 Partition 使用，可以流水线执行，所以不需要切 Stage。宽依赖如 reduceByKey、groupByKey、join 通常需把相同 key 的数据重新分布到同一个 Partition，会发生 Shuffle。Shuffle 是一个天然边界，上游需要先写出 shuffle 数据，下游再读取这些数据。因此 Spark 会在宽依赖处切 Stage。简单记：窄依赖不断，宽依赖切 Stage。

9. 为什么 join 通常是宽依赖？什么情况下 join 可以避免 Shuffle？

Join 的本质是把两个 RDD 中相同 key 的数据放到一起。默认情况下，两个 RDD 的相同 key 不一定在同一个 Partition，所以 Spark 需要通过 Shuffle 重新分区，把相同 key 拉到一起，因此 join 通常是宽依赖。但如果两个 RDD 已经使用相同的 Partitioner 按相同 key 分区，并且 Partition 数和分区规则一致，那么相同 key 已经位于对应分区中，join 时就可以避免额外 Shuffle。这种情况下 join 可以表现为窄依赖或至少不会新增 Shuffle。

10. MapReduce 和 Spark 的容错机制有什么相同点和不同点？

二者都依赖“可重算”思想来实现容错。MapReduce 的输入数据通常存储在 HDFS 中，Task 失败后可以重新启动 Task，从原始输入或中间结果重新计算。Spark 的 RDD 记录 lineage，也就是 RDD 是从哪些父 RDD 经过哪些 Transformation 得到的，如果某个 Partition 丢失，可以沿着 lineage 重新计算。不同点是，MapReduce 更偏向 Job/Task 级别重跑，中间结果常落盘；Spark 可以在 Partition 粒度通过 lineage 重算，窄依赖恢复成本较低，但宽依赖涉及 Shuffle 时恢复成本更高。

11. Dremel 为什么需要 r/d？它解决了列式存储嵌套数据的什么问题？

Dremel 面向嵌套数据的交互式分析。列式存储可以只读取查询需要的列，减少 I/O，但嵌套记录被拆成列后，会丢失原本的层级结构信息。r，也就是 repetition level，用来表示当前值从哪一层 repeated 字段开始重复；d，也就是 definition level，用来表示字段路径实际定义到哪一层。通过给每个列值附加 r/d，Dremel 可以在只读取部分列的情况下恢复嵌套结构的投影。简单说，r/d 是为了让列式存储既能省 I/O，又不丢嵌套结构。

12. NoSQL 为什么常常牺牲强一致性？CAP 和 BASE 怎么解释这个选择？

在分布式系统中，网络分区很难完全避免，所以 CAP 中的 P 通常必须考虑。当发生网络分区时，系统如果坚持强一致性，可能需要拒绝部分请求，导致可用性下降；如果坚持可用性，就可能允许不同副本短时间不一致。很多 Web 场景，如点赞数、Feed、缓存、日志统计，可以容忍短暂不一致，因此 NoSQL 系统常选择 AP，牺牲强一致性换取高可用和高扩展。BASE 的基本可用、软状态、最终一致性正是这种思想：系统允许暂时不一致，但最终会收敛。

13. HBase 的 LSM-tree 和 Compaction 分别解决什么问题？又带来什么代价？

LSM-tree 思想主要解决高并发写入问题。HBase 写入时先写 WAL，再写 MemStore，MemStore 满后顺序 flush 成 HFile。这样避免了频繁随机写磁盘，提高写入吞吐。但随着写入增加，磁盘上会产生多个 HFile，读取时可能需要查 MemStore、BlockCache 和多个 HFile，读放大变大。Compaction 用来合并多个 HFile，减少文件数量，清理删除标记和过期版本，提高读取效率。但 Compaction 本身会消耗磁盘 I/O 和 CPU，可能影响线上性能。所以 LSM-tree 提升写性能，Compaction 用来控制读放大和存储碎片。

14. Flink 的 Checkpoint 如何保证有状态计算故障后能恢复？

Flink 的 Checkpoint 是一致性状态快照机制。系统周期性地为各个有状态算子保存状态，并记录数据源消费位置，例如 Kafka offset。当任务失败时，Flink 会重启作业，从最近一次成功完成的 Checkpoint 恢复每个 subtask 的状态，同时把 source 的读取位置恢复到对应 offset。这样程序可以从一个一致的历史点继续处理，避免状态和输入位置不匹配。配合可重放数据源和支持事务或幂等的 Sink，Flink 可以实现 exactly-once 语义。

15. 如果一道综合题让你为“大规模用户行为分析平台”选择技术栈，可以怎么答？

可以分层设计。底层用 HDFS 存储海量原始日志，因为它适合大文件顺序写入和离线批处理。离线统计和历史分析可以用 MapReduce 或 Spark，其中 Spark 更适合复杂 DAG、迭代计算和交互分析。需要按用户实时查询画像或索引时，可以把结果写入 HBase，利用 RowKey 支持随机读写。实时行为流、窗口统计和异常检测可以用 Flink，利用 Event Time、Watermark、State 和 Checkpoint 处理乱序与容错。如果系统对高可用和水平扩展要求高，并能接受短暂不一致，可以结合 NoSQL 和 BASE 思想。这样的架构分别解决存储、离线计算、实时计算和在线查询问题。

技术栈选择

1. 为“大规模电商用户行为分析平台”选择技术栈

可以分层设计。底层用 **HDFS** 存储原始点击日志、订单日志、搜索日志，因为这些数据量大、适合大文件顺序写入和离线批处理。离线统计可以用 **MapReduce** 或 **Spark**，其中 MapReduce 适合简单稳定的批处理任务，Spark 更适合复杂 DAG、迭代计算和交互式分析，比如用户画像、商品推荐特征统计。

如果需要按用户 ID 快速查询画像结果，可以把计算后的结果写入 **HBase**，利用 RowKey 支持随机读写。实时点击流、实时下单监控、异常流量检测可以用 **Flink**，因为它支持 Event Time、Watermark、窗口、状态和 Checkpoint；如果只是简单低延迟处理，也可以用 **Storm** 做实时日志解析和告警。

对于复杂嵌套日志，比如商品曝光、点击、推荐位、设备信息等嵌套结构，可以借鉴 **Dremel** 的列式存储思想，只读取查询需要的列，提高交互式分析效率。用户购物车、缓存、会话状态等可以用 **NoSQL** 的 key-value 或 document model，利用 BASE 和最终一致性换取高可用。整体上，HDFS 管原始存储，Spark/MapReduce 管离线计算，Flink/Storm 管实时流，HBase/NoSQL 管在线查询，Dremel 思想用于嵌套数据分析。

2. 为“社交平台 Feed 流与实时热点分析系统”选择技术栈

社交平台的数据包括用户关系、帖子内容、Feed 流、点赞评论、实时热点等。原始行为日志和历史帖子数据可以先存入 **HDFS**，便于长期保存和离线分析。每天离线统计用户活跃度、关系链特征、热门内容，可以使用 **Spark**，因为 Spark 比 MapReduce 更适合多轮迭代和复杂数据处理；简单批量统计任务也可以用 **MapReduce** 完成。

Feed 流在线查询适合使用 **HBase**。例如 `post_content` 存帖子正文，`user_inbox` 存收件箱索引，`user_outbox` 存用户发件箱，RowKey 可以用 `user_id`，列名用倒序时间戳加 `post_id`，方便读取最近 N 条动态。普通用户可以写扩散，大 V 可以采用读扩散或混合推拉。

实时热点、实时点赞数、异常评论检测可以用 **Flink**，因为它支持有状态窗口统计、Watermark 和 Checkpoint。若只做简单流式转发、过滤和计数，也可以用 **Storm** 的 Spout/Bolt 拓扑实现。帖子内容、用户资料这类半结构化数据可以使用 **NoSQL document model**，关系推荐也可以考虑 graph model。若平台日志是嵌套结构，比如一次请求里包含多个曝光、点击、推荐原因，可以用 **Dremel** 的 r/d 和列式思想做交互分析。整体设计体现了存储、离线分析、实时计算、在线查询和一致性取舍。

3. 为“物联网传感器监控与异常检测平台”选择技术栈

物联网平台会持续产生大量传感器数据，例如设备 ID、时间戳、温度、压力、状态码等。原始历史数据可以存入 **HDFS**，因为 HDFS 适合持续追加的大文件和后续批处理分析。长期趋势分析、设备历史统计、模型训练特征生成可以使用 **Spark**，如果只是周期性批量统计，也可以用 **MapReduce**。

在线查询某个传感器某段时间的数据，适合用 **HBase**。RowKey 可以设计成 `sensor_id#reverse_timestamp`，这样同一设备的数据会聚集在一起，并支持按时间范围 Scan。设备状态、配置、最近一次上报值也可以用 **NoSQL key-value/document store** 保存，牺牲部分强一致性换取高可用和高吞吐。

实时异常检测更适合 **Flink**，因为传感器数据可能乱序到达，需要 Event Time 和 Watermark；同时连续升温、连续异常、滑动窗口平均值都需要 Keyed State 和 Checkpoint。简单阈值报警也可以用 **Storm** 实现，结构是 MetricSpout → ParseBolt → FilterBolt → AlertBolt。若传感器上报数据是嵌套结构，例如一个设备一次上报多个子传感器数组，可以借鉴 **Dremel** 的列式编码和 r/d 拼回方式，支持只查询部分字段。这个系统里，HDFS 保存历史，HBase 支持在线时间范围查询，Flink 负责实时异常，Spark/MapReduce 做离线分析，NoSQL 支撑高可用配置和状态存储。

4. 为“搜索引擎日志分析与倒排索引系统”选择技术栈

搜索系统会产生查询日志、点击日志、网页抓取内容和倒排索引数据。网页原始数据和搜索日志可以存入 **HDFS**，因为数据规模大，适合顺序写入和离线批处理。构建倒排索引可以用 **MapReduce**：Map 输出 `<word, doc_id>`，Shuffle 按 word 聚合，Reduce 输出 `<word, list(doc_id)>`。如果索引构建流程包含多轮清洗、排序、特征计算，使用 **Spark** 更高效，因为 Spark 可以把多个 Transformation 组织成 DAG，并缓存中间结果。

在线索引或部分查询结果可以写入 **HBase**，例如 RowKey 为 `word`，value 为文档列表或索引块，支持按关键词快速查询。搜索热词、实时点击率、异常流量检测适合用 **Flink** 做事件时间窗口统计；简单日志流处理可以用 **Storm** 进行实时过滤和计数。

网页内容和搜索结果可能包含复杂嵌套字段，如标题、正文、链接、锚文本、结构化摘要等，交互式分析时可以借鉴 **Dremel** 的列式存储思想，只扫描需要的字段。搜索系统中的缓存、用户搜索会话、推荐候选集可以用 **NoSQL** 保存，利用高并发读写和最终一致性提高可用性。整体上，MapReduce/Spark 负责索引构建和离线统计，Flink/Storm 负责实时监控，HBase/NoSQL 提供在线查询和缓存，HDFS 是底层数据湖。

5. 为“金融风控与交易监控平台”选择技术栈

金融风控平台既需要保存海量历史交易，又需要实时发现异常交易。底层可以用 **HDFS** 保存历史交易流水、登录日志、设备指纹和风控结果，作为长期数据湖。离线建模、用户风险画像、历史规则回溯可以使用 **Spark**，因为 Spark 支持复杂 DAG、迭代计算和缓存；简单的批量清洗、统计也可以用 **MapReduce**。

在线风险画像可以存入 **HBase**，以 `user_id`、`account_id` 或 `card_id` 作为 RowKey，支持风控服务低延迟读取用户历史特征。实时交易流更适合使用 **Flink**，因为风控需要按事件时间处理乱序数据，并维护用户最近 N 分钟交易次数、金额总和、异常设备数等 Keyed State；Checkpoint 可以保证故障恢复时状态不丢。若只是简单实时规则过滤，也可以用 **Storm** 搭建 Spout/Bolt 拓扑。

交易日志常包含嵌套字段，如用户、设备、商户、地理位置、支付方式、风控规则命中列表等。做交互式审计分析时，可以借鉴 **Dremel** 的列式存储和 r/d 思想，提高嵌套字段查询效率。至于 **NoSQL**，可以用于缓存风险规则、设备画像、会话信息等；但金融核心交易更强调强一致性，不能随意用最终一致性替代 ACID。因此这道题要写清楚：NoSQL/BASE 可以用于非核心缓存和画像，核心交易仍应保证强一致。这个答案体现了 HDFS 存

储、Spark/MapReduce 离线计算、Flink/Storm 实时监控、HBase 在线画像、Dremel 嵌套分析和 NoSQL 一致性取舍。

具体场景分析

对，我也觉得他如果偏应用和综合，不会单问“什么是 HDFS / 什么是 RDD”这种死概念，更可能给你一个场景，让你判断：**用什么系统、为什么、怎么设计、哪里会出问题**。你准备时就按“场景题模板”去背，性价比最高。🔥 我们资料里的答题准则本来也强调：遇到设计/伪代码题要写清楚输入、输出、key、数据流动、为什么这样分组/分区/连接；对比题要服务考试场景，不要泛泛比较。

下面是他可能出的综合简答方向。

1. 给一个业务场景，让你选技术栈

比如题目可能问：

“某电商平台每天产生大量点击日志，需要离线分析、实时热点统计、用户画像查询，请设计大数据平台。”

你就按这个套路写：

HDFS：存原始大文件日志。
MapReduce / Spark：做离线批处理，Spark 更适合复杂 DAG 和迭代分析。
HBase：存用户画像、查询结果，支持 RowKey 随机读写。
Flink / Storm：做实时流处理，Flink 更适合有状态、乱序、窗口统计。
NoSQL：存缓存、会话、半结构化数据，接受最终一致性。
Dremel：如果日志是嵌套结构，可用列式思想做交互分析。

核心不是把所有技术都堆上去，而是写出：**每一层解决什么问题**。

2. 给一个“海量小文件”场景，问为什么 HDFS 不合适，怎么改

比如：

“用户上传大量头像、小图、短视频缩略图，能不能直接存 HDFS？”

答法：

HDFS 不适合海量小文件，因为每个文件和 Block 都需要 NameNode 维护元数据，小文件数量太多会占用大量 NameNode 内存；同时每个小文件数据量小，调度和寻址开销反而比数据本身更重，无法发挥大 Block 顺序读写优势。可以通过小文件合并、打包成逻辑对象、使用对象存储或缓存/CDN 等方式优化。HDFS 更适合大文件顺序读写，不适合小文件随机访问。

这个特别像应用题，因为 HDFS 课件里专门有海量小文件管理案例。

3. 给日志统计任务，让你比较 MapReduce 和 Spark

可能问：

“同样是日志 PV/UV 统计，为什么 Spark 往往比 MapReduce 快？”

答法：

MapReduce 每轮任务通常要读写 HDFS，中间结果落盘，多个 Job 串联时磁盘 I/O 和调度开销大。Spark 使用 RDD 和 DAG，把多个 Transformation 连成执行图，遇到 Action 才触发执行；窄依赖可以流水线执行，宽依赖处才 Shuffle；中间结果可以 cache 到内存，因此适合多轮迭代和复杂数据处理。Spark 原理 PPT 里也明确强调从逻辑 DAG 到物理计划要划分 Stage 和 Task。

4. 给“实时监控/异常检测”场景，让你比较 Storm、Spark、Flink

可能问：

“服务器指标流需要实时报警，还要按事件时间统计窗口异常次数，选 Storm、Spark 还是 Flink？”

答法：

Storm 适合简单低延迟流处理，Topology 常驻运行，Spout 产生数据，Bolt 做解析、过滤、报警。Spark Streaming 是微批，吞吐高但实时性不如原生流。Flink 更适合复杂有状态流处理，因为它支持 Event Time、Watermark、Window、Keyed State 和 Checkpoint，能够处理乱序事件并保证故障恢复。所以简单阈值报警可用 Storm；涉及乱序、窗口、状态、一致性恢复时优先 Flink。Storm 的定位就是分布式实时流式计算，适合异常监测、实时统计等场景。

5. 给 Feed 流/社交平台，让你设计 HBase 表

可能问：

“设计微博/朋友圈 Feed 流，如何用 HBase 存储？”

答法一定要从访问模式出发：

```
post_content: RowKey = post_id, 存帖子正文。
user_outbox: RowKey = author_id, 列名 = reverse_timestamp#post_id。
user_inbox: RowKey = receiver_id, 列名 = reverse_timestamp#post_id。
follower_list: RowKey = author_id, 存粉丝。
followee_list: RowKey = user_id, 存关注列表。
```

然后解释：

普通用户可以写扩散，发帖时把 post_id 写入粉丝 inbox，读 Feed 很快；大 V 粉丝太多，写扩散压力大，可以用读扩散或混合推拉，只推活跃粉丝，读的时候再拉取大 V outbox。HBase 适合这个场景，是因为它支持按 RowKey 的随机读写和按时间范围 Scan。

6. 给传感器/时序数据，让你设计 RowKey

可能问：

“海量传感器不断上报数据，如何用 HBase 存储并查询某设备最近一小时数据？”

答法：

RowKey 可以设计成 `sensor_id#reverse_timestamp` 或 `sensor_id#timestamp`。如果主要查某设备最近数据，就把 `sensor_id` 放前面，让同一设备数据聚集；时间放后面，支持范围 Scan。用倒序时间戳可以让最新数据排在前面。列族可以放 `data`，列包括 `value`、`status`、`type`。需要注意热点问题，如果某些设备写入特别集中，可以加盐或分桶。

这类题本质考 HBase：先分析访问模式，再设计 RowKey。

7. 给“乱序点击流”，问 Flink Watermark 怎么用

可能问：

“用户点击日志乱序到达，要按事件时间统计 5 分钟窗口 PV，如何设计？”

答法：

使用 Flink 的 Event Time 语义，从日志中提取事件真实发生时间，并生成 Watermark。Watermark(t) 表示系统认为时间戳小于等于 t 的数据基本已经到达，当 Watermark 超过窗口结束时间时触发窗口计算。然后按 URL `keyBy(url)`，使用 5 分钟滚动窗口，聚合 count。乱序数据在允许延迟范围内仍可进入正确窗口，超过 Watermark 太多的数据成为 late data。资料里也强调乱序流下优先讲 Event Time + Watermark。

8. 给“状态恢复”场景，问 Checkpoint / Savepoint

可能问：

“Flink 作业统计每个用户最近 10 分钟消费金额，任务失败后如何恢复？”

答法：

Flink 会周期性执行 Checkpoint，把每个 stateful subtask 的状态保存到外部持久化存储，同时记录 source 的消费位置，如 Kafka offset。故障后，作业从最近一次成功 Checkpoint 恢复所有状态，并把 source 位置恢复到对应 offset，继续处理数据。Checkpoint 主要用于自动容错；Savepoint 是用户手动触发，常用于升级、迁移、修改并行度或暂停恢复。资料里这两个区别就是：checkpoint 自动容错，savepoint 人工运维。

9. 给嵌套日志，问为什么 Dremel 适合

可能问：

“日志中一条记录包含多个广告曝光、点击、设备信息、推荐理由等嵌套字段，为什么 Dremel 适合交互分析？”

答法：

嵌套数据如果按行存储，查询某几个字段也要读整条记录，I/O 浪费。Dremel 使用列式存储，把嵌套字段拆成列，只读取查询需要的列。同时用 repetition level 和 definition level 保存嵌套结构：r 表示从哪层 repeated 字段开始重复，d 表示路径实际定义到哪一层。这样既能减少读取数据量，又能从部分列恢复嵌套记录的投影。Dremel PPT 中明确强调它要解决“保留结构，并能从字段子集重构”的问题。

10. 给一致性/高可用场景，问 NoSQL、CAP、BASE 怎么取舍

可能问：

“点赞数、评论数、购物车、金融交易分别应该如何考虑一致性？”

答法：

点赞数、评论数、Feed 排序这类场景通常可以容忍短暂不一致，适合 NoSQL 的 BASE 思想，用基本可用、软状态、最终一致性换取高可用和高扩展。购物车需要较高可用，也可以采用最终一致性加补偿机制。金融核心交易、库存扣减、支付记账则不能随意牺牲一致性，应更强调 ACID 和强一致。CAP 中，分布式系统通常必须考虑分区容忍性 P，因此常在 C 和 A 之间取舍；Web 场景常偏 AP，金融核心场景更偏 CP 或强事务。

11. 给“系统瓶颈”，让你指出问题和优化

比如：

“一个 Spark 作业很慢，可能原因有哪些？”

你可以答：

可能是发生了大量 Shuffle，如 `groupByKey`、`join`、`distinct`；可能是数据倾斜，少数 key 数据过多；可能是没有缓存复用 RDD，导致多个 Action 反复重算；可能是 partition 数设置不合理，Task 太少导致并行度低，Task 太多导致调度开销大；也可能是输入小文件太多，导致 task 过碎。优化包括用 `reduceByKey` 替代 `groupByKey`、合理设置 shuffle partition、cache 复用 RDD、处理热点 key、合并小文件。

12. 给“一个系统为什么不适合某场景”

这种也很可能出，因为他偏应用就爱问“为什么不用 X”。

比如：

“为什么不用 MapReduce 做实时告警？”

答法：

MapReduce 是批处理模型，输入通常是静态数据集，任务启动、调度、Map、Shuffle、Reduce 都有较大开销，中间结果常写磁盘或 HDFS，适合高吞吐离线计算，不适合毫秒级响应。实时告警要求数据持续到达、持续计算、低延迟输出，更适合 Storm 或 Flink 这类流处理系统。流处理课程里也明确说数据流的特点是无限、连续、实时响应，MapReduce 无法很好处理。

13. 给“批流一体”，问 Spark 和 Flink 思路区别

可能问：

“Spark 和 Flink 都能做批流处理，它们的统一思想有什么不同？”

答法：

Spark 本质是批处理引擎，Spark Streaming 把流切分成一系列小批次，因此是用批处理模拟流处理。Flink 本质是流处理引擎，它把批处理看成有限流，把有界数据看成无界流的特殊情况。因此 Spark 在批处理和迭代计算中优势明显，而 Flink 在低延迟、事件时间、状态、乱序处理和 exactly-once 方面更自然。资料中也直接总结：Spark 把流处理看成特殊批处理，Flink 反过来把批处理看成有限流。

14. 给“在线查询 + 离线分析 + 实时计算”的混合架构

可能问：

“为什么一个大数据平台往往不是只用一个系统，而是多个系统组合？”

答法：

不同系统解决的问题不同。HDFS 适合低成本可靠存储海量原始数据；MapReduce/Spark 适合离线批处理，其中 Spark 更适合复杂 DAG 和迭代任务；HBase 适合按 RowKey 的在线随机读写；Storm/Flink 适合实时流处理，其中 Flink 更适合有状态和事件时间；NoSQL 适合高并发、高可用、半结构化数据；Dremel 适合嵌套数据的交互式列式分析。单一系统很难同时满足低成本存储、复杂计算、实时响应、随机查询和一致性要求，所以实际平台通常采用分层组合。

15. 给“同一个任务怎么用不同系统实现”，让你比较

比如：

“统计每个 URL 的访问次数，可以用 MapReduce、Spark、Storm、Flink 分别怎么做？”

答法：

MapReduce：离线统计，Map 输出 `<url,1>`，Shuffle 按 URL 分组，Reduce 求和，适合历史日志批处理。
Spark：读取 HDFS 日志，`map -> reduceByKey -> save`，可以利用内存和 DAG，提高复杂批处理效率。
Storm：日志流进入 Spout，ParseBolt 解析，CountBolt 按 URL 统计，`fieldsGrouping(url)`，适合低延迟实时计数。
Flink：Source 读取日志流，提取 Event Time 和 Watermark，`keyBy(url)` 后按窗口聚合，适合乱序和有状态窗口统计。

这题一看就是综合应用，核心是写出 **离线批处理 vs 实时流处理 vs 有状态窗口** 的区别。

你现在最该准备的不是再背散点，而是背这句万能开头：

这个场景可以按“存储层、离线计算层、实时计算层、在线查询层、一致性取舍”来设计。**HDFS** 存原始大数据，**MapReduce/Spark** 做离线计算，**Storm/Flink** 做实时流，**HBase/NoSQL** 做在线访问，**Dremel** 思想适合嵌套数据交互分析。